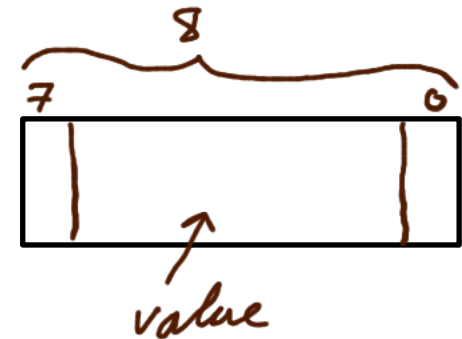


- 8051 has CPU: The CPU contains two types of Registers

① 8-bit registers : A, B, R₀, R₁, R₂, R₃, R₄, R₅, R₆, R₇, SP,
PSW.



Range of values:

min		max
<u>0000</u> <u>0000</u> B	→	1111 1111 B
00 H	→	FF H
0	→	255 D

② 16-bit registers : PC, DPTR



Range of values

min		max
0000 H	→	FFFF H
0 D	→	65535 D

- All previous registers are called the most widely used registers. (2)
-

* $A = ACC = \text{Accumulator (8bit)}$

- It is must be used in all arithmetic operations
(+, -, $\times$, $\div$)
 - In general, it is used in logical operations (AND, OR, ...)
 - Must be used in external data transmission.
-

* $B \rightarrow \text{Auxiliary register}$

- It is used as an auxiliary register with A in arithmetic operation.

① Multiplication ($\times$)
 before $\rightarrow$ hold second operand
 after $\rightarrow$ hold high part of result.

② Division
 before $\rightarrow$ hold second operand (divisor)
 after $\rightarrow$ hold the remainder
 باقی

* $R_0 \rightarrow R_7$: general purpose registers

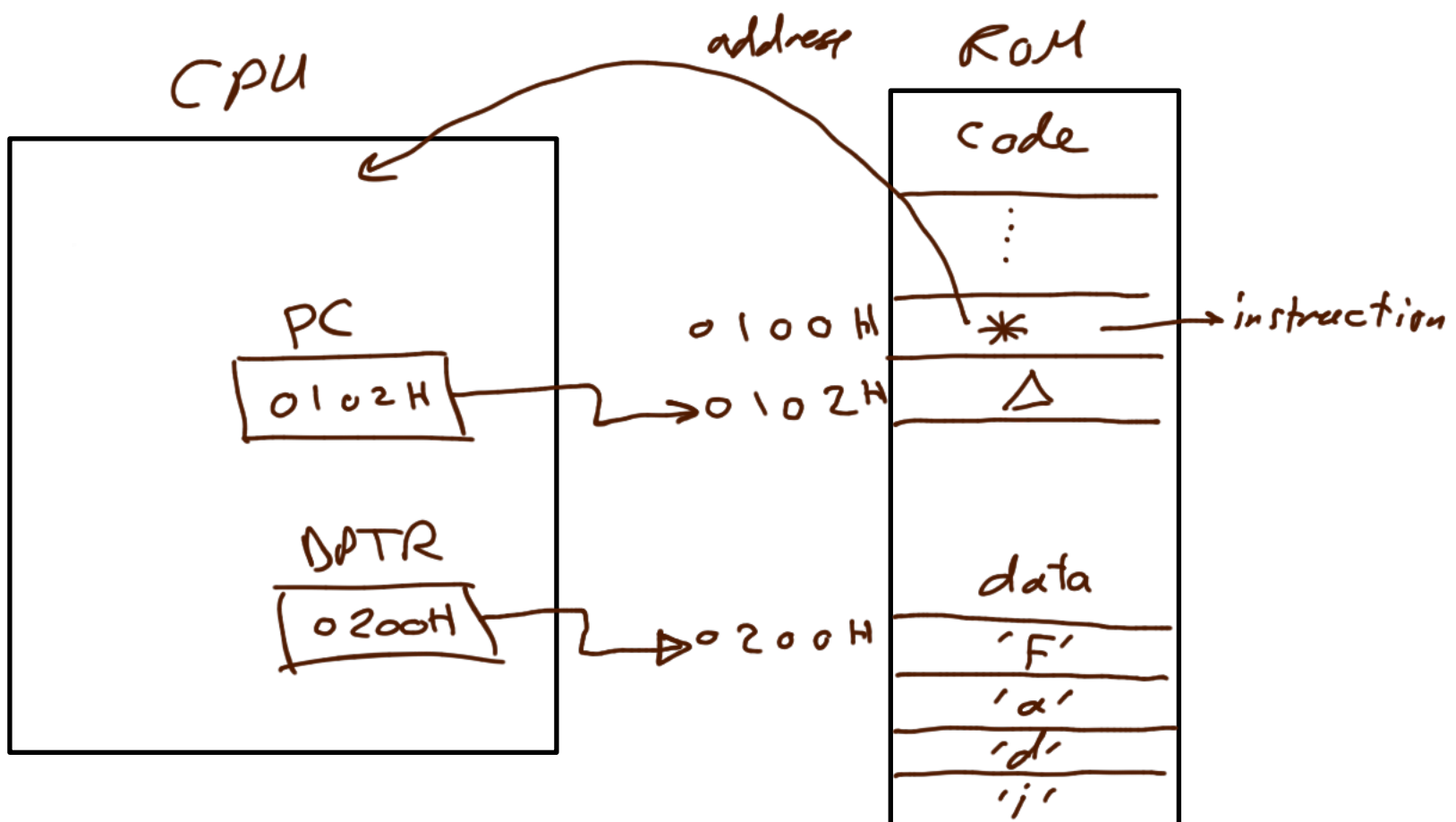
(3)

* PC: Program counter (16 bit)

- Is used to point to the address of the next instruction that should be fetched from (ROM) memory to be executed in CPU.

* DPTR: Data pointer (16 bit)

- Is used to point to the address of data that should be fetched from memory to the CPU



- The max size of memory that can be connected with ④

8051: 64KB

capacity

code memory (ROM)

Data memory (external)

$$2^{16} = 2^6 \times 2^{10} = \underline{64KB}$$

PC $\rightarrow$ 16 bit

		(ROM)	program	
address	opcode			
1 0000			ORG 0H	;start at location 0
2 <u>0000</u>	<u>7D25</u>		<u>MOV R5, #25H</u>	;load 25H into R5
3 $\rightarrow$ 0002	7F34		MOV R7, #34H	;load 34H into R7
4 0004	7400		MOV A, #0	;load 0 into A
5 0006	2D		ADD A, R5	;add contents of R5 to A
				;now A = A + R5
6 0007	2F		ADD A, R7	;add contents of R7 to A
				;now A = A + R7
7 0008	2412		ADD A, #12H	;add to A value 12H
				;now A = A + 12H
8 000A	80FE HERE:		SJMP HERE	;stay in this loop
9 000C			END	;end of asm source file

* PSW: program status Register (8 bit)

5

- is used to reflect the status of CPU after executing some instructions.

- Each bit in PSW is called flag bit.

7	6	5	4	3	2	1	0
Cy	AC	/	RS1	RS0	OV	-	P

① PSW.7 = C = Cy

- is carry flag

- is set (1) when there is carry out from D7.

↓
addition

subtraction: carry in D7
borrow

MOV A, #93H

ADD A, #0E2H

PSW.7 = C = 1

PSW.6 = AC = 0

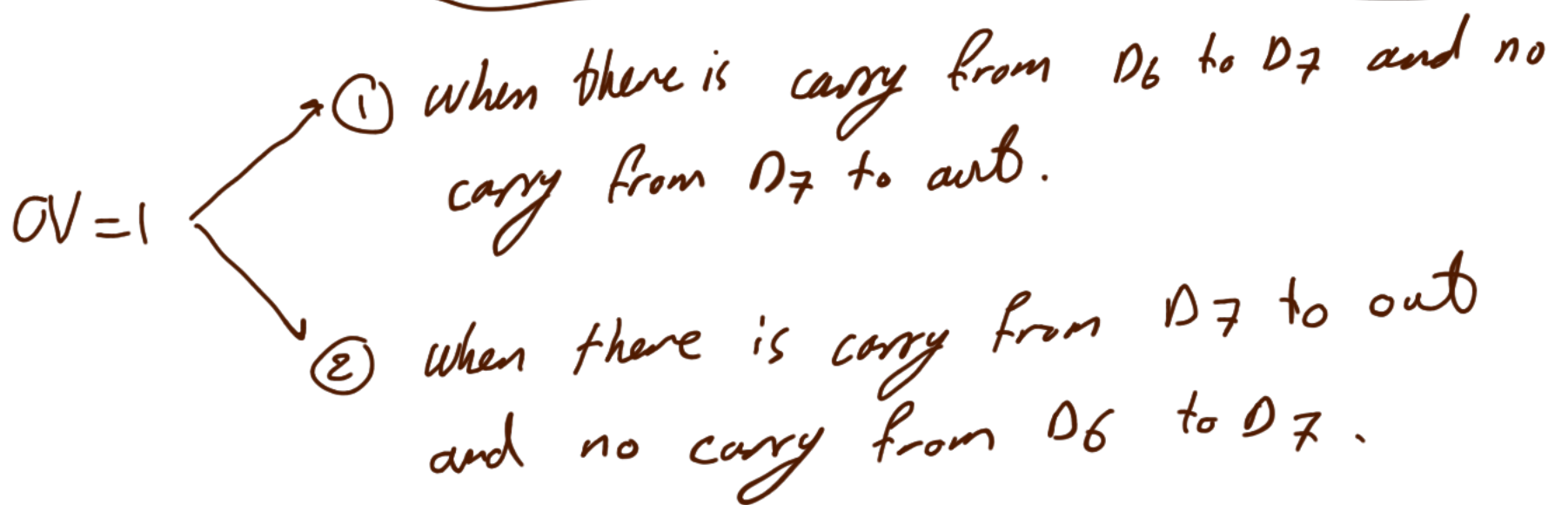
	D ₇	P ₄	P ₃	D ₀
	1	0	0	1
+	1	1	1	0
	0	1	1	1

② $PSW.6 = AC$

- is called auxiliary carry flag (half carry)
 - is set (1) when there is carry from D₃ to D₄
 - is used only for BCD addition
-

③ $PSW.2 = OV$

- is called overflow flag
 - is set (1) to indicate that the result of signed arithmetic operation is error (out of the range). Does not fit in the range.
-



MOV A, #93H
ADD A, #8EH

Different

PSW.7 = CY = 1

PSW.6 = AC = 1

PSW.2 = OV = 1

PSW.0 = P = 0

07 06 05 04 03 02 01 00
1 0 0 1 0 0 1 1

+ 1 0 0 0 1 1 1 0

← 0 0 1 0 0 0 0 1

④ PSW.0 = P

is parity flag

is used to detect errors

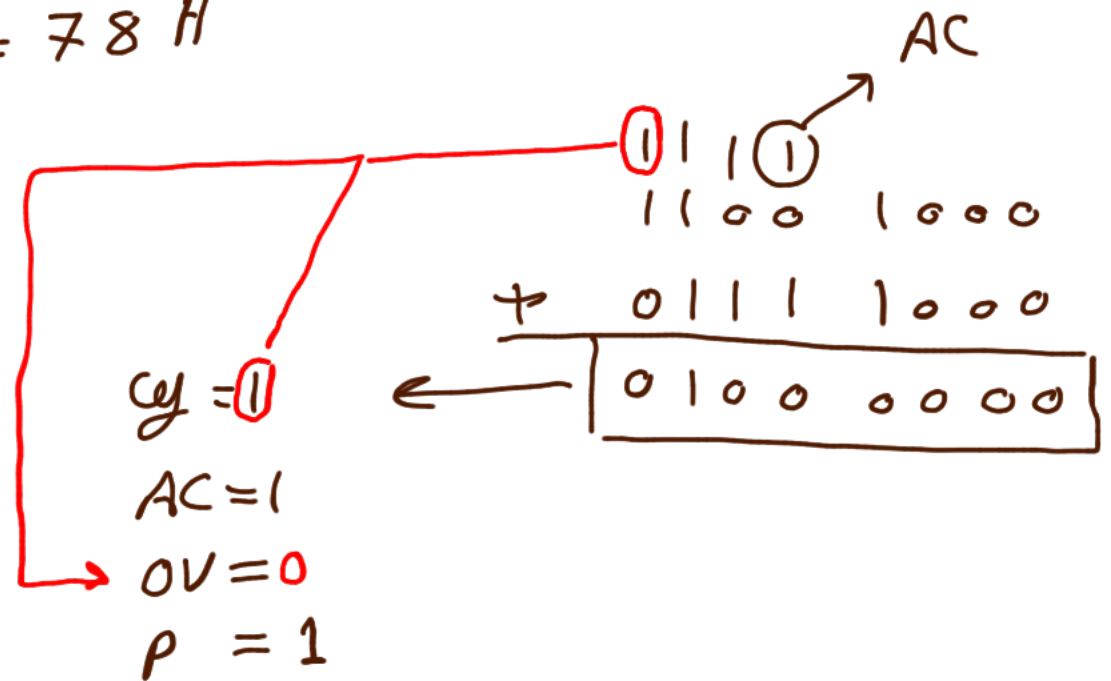
is used to reflect number of ones (1's) in
A (Accumulator).

P → = 1 if # of 1's in A is odd
→ = 0 " " " " " " A is even

show me the content of PSW (CY, AC, OV, P) ⑤
after executing the following

MOV A, #0C8H

ADD A, #78H



ADD A, source ; A = A + source

destination
8 bit

- affects PSW flags (CY, AC, P, OV)

ADD B, #25H X

ADD A, B ✓

ADD B, A X

* 8051 features

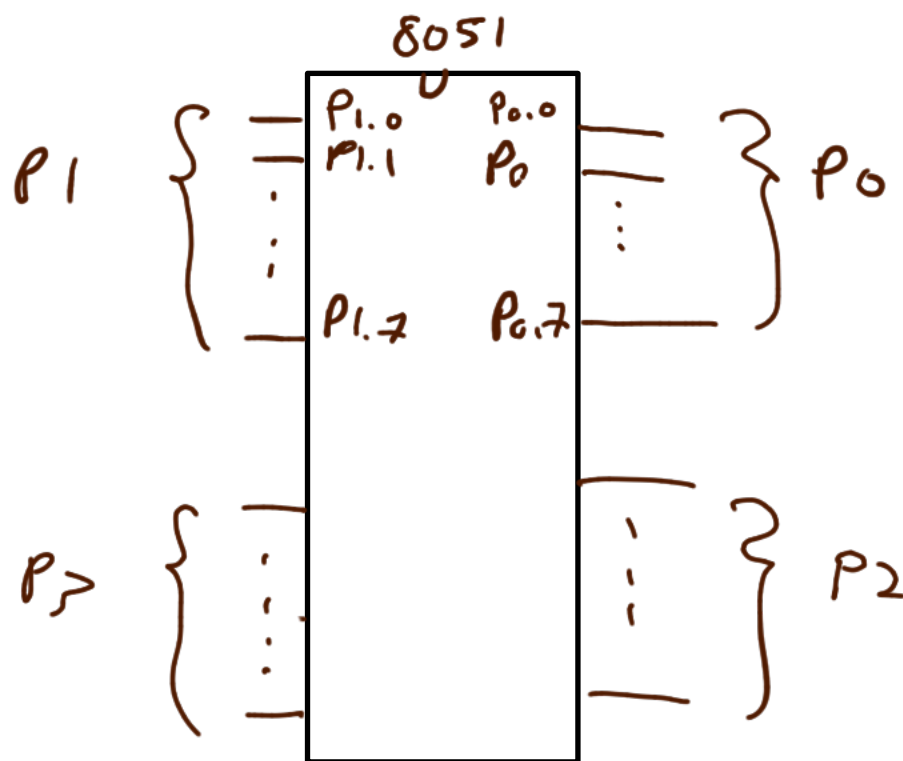
9

Table 1-4: Comparison of 8051 Family Members

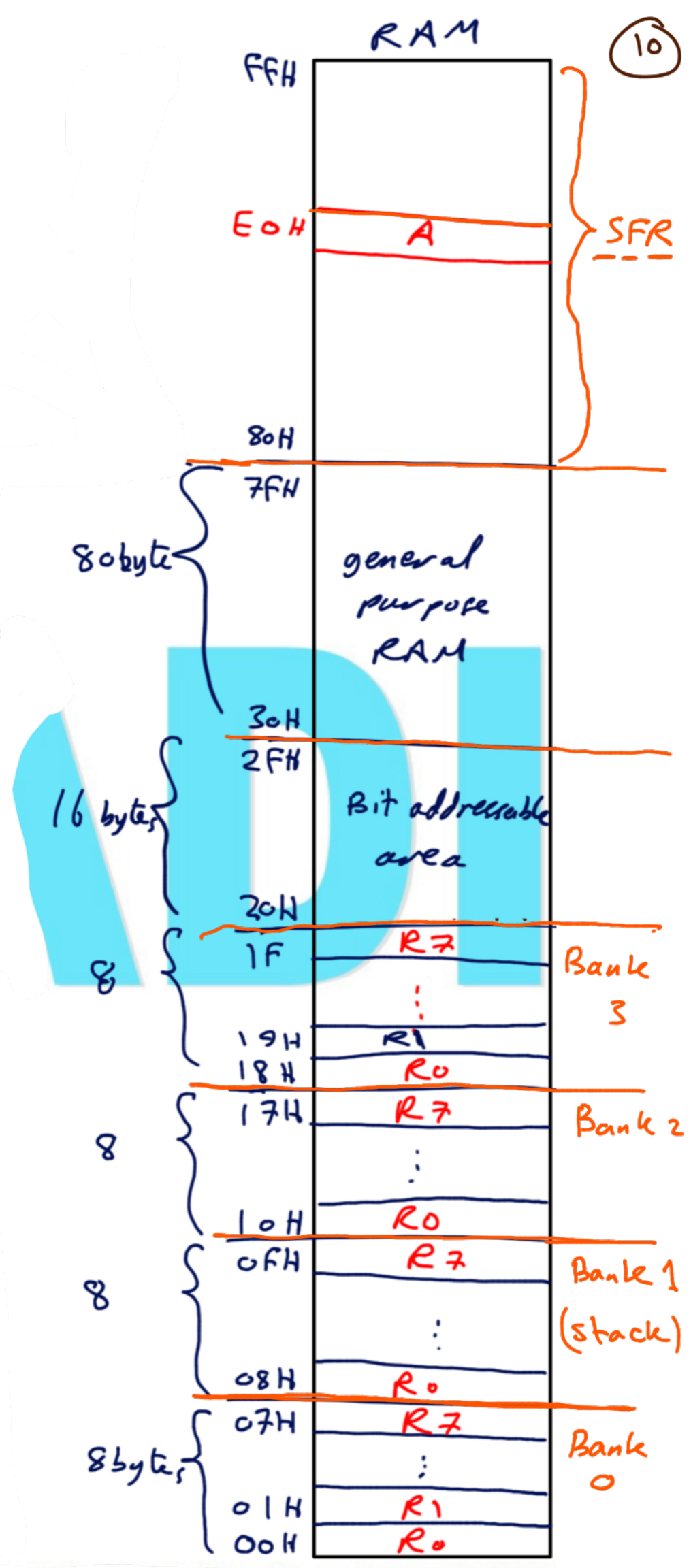
Feature	8051	8052	8031
ROM (on-chip program space in bytes)	4K	8K	0K
RAM (bytes)	128	256	128
Timers	2	3	2
I/O pins (4 I/O ports)	32	32	32
Serial port	1	1	1
Interrupt sources	6	8	6

each port is 8 pins

$$8 \times 4 = 32 \text{ pin}$$



- 8051 is an 8-bit microcontroller



- Up on reset (power on) all registers ($R_0 \rightarrow R_7$)

will be in bank 0 ($00H \rightarrow 07H$)

- programmer can move ($R_0 - R_7$) from bank 0 to bank 1, 2, or 3, using PSW.4 (RS1) and PSW.3 (RS0)

RS1 PSW.4	RS0 PSW.3	Bank
0	0	0 by default
0	1	1
1	0	2
1	1	3

- Move ($R_0 - R_7$) from bank 0 to bank 3.

SETB PSW.4 ; PSW.4 = RS1 = 1

SETB PSW.3 ; PSW.3 = RS0 = 1

in bank 3

SETB Bit ;	Bit = 1
------------	---------

CLR Bit ;	Bit = 0
-----------	---------

Write a program to move the content of R1 from bank 3 to R7 in to bank 1

ORG 0000H

SETB PSW.3

SETB PSW.4 ; in bank 3

MOV A, R1

SETB PSW.3

CLR PSW.4 ; in bank 1

MOV R7, A

SJMP \$

END
